

Comprehensive Architecture and Utilization Plan for the Yosys Open Synthesis Suite

The Vanguard of Open-Source Electronic Design Automation

The realm of Electronic Design Automation (EDA) has historically operated under a paradigm dominated by proprietary, closed-source toolchains. This environment has consistently imposed high barriers to entry for academic research, custom algorithmic development, and the pursuit of fully transparent hardware compilation. The Yosys Open SYnthesis Suite represents a foundational disruption to this established ecosystem. Originally conceptualized and developed in late 2012 as a bachelor's thesis project aimed at supporting synthesis for a coarse-grained reconfigurable architecture (CGRA), the framework has rapidly evolved into an expansive, vendor-independent Register Transfer Level (RTL) synthesis infrastructure.¹ Today, it is widely regarded within the industry and academia as the "GCC of hardware synthesis," serving as a critical foundational component in fully open-source, end-to-end silicon compilers.² It acts as the primary synthesis engine for the OpenLane flow, which targets the SkyWater 130nm node for Application-Specific Integrated Circuit (ASIC) design, and it operates seamlessly alongside place-and-route tools like nextpnr to program modern Field-Programmable Gate Arrays (FPGAs) such as the Lattice iCE40 and ECP5 series.²

The architecture of Yosys is explicitly modular, designed to closely mirror the multi-stage compilation methodologies of modern software compilers.⁴ The overarching control flow operates by invoking a sequence of discrete subsystems. It begins with high-level frontends that parse hardware description languages, proceeds through intermediate optimization passes that minimize boolean logic, moves into technology mapping engines that anchor the logic to physical silicon primitives, and finally concludes with backends that export the finalized netlist.⁴ This strict modularity permits an unprecedented level of customizability. Hardware engineers can orchestrate these existing algorithmic passes using synthesis scripts, typically structured in .ys or .tcl formats, to adapt the tool to any specific synthesis job.⁵ Furthermore, when pre-existing algorithms are insufficient, developers can extend the core framework by injecting custom C++ passes directly into the execution graph.⁵

At the nucleus of the Yosys engine is a sophisticated intermediate data structure known as the RTL Intermediate Language (RTLIL). When source code is ingested—most notably the synthesizable subset of Verilog-2005, which enjoys robust and extensive native support—the frontend parsers initially generate an Abstract Syntax Tree (AST).⁵ This language-agnostic AST is subsequently elaborated and compiled into the highly structured RTLIL format.⁴ From that

point forward, all transformation matrices, resource sharing routines, structural equivalence checking algorithms, and technology mappings operate strictly on the RTLIL data structures in memory.⁴ The output backend then transcribes this finely tuned RTLIL graph into the desired target format, which ranges from structural gate-level Verilog to specialized formats like BLIF, EDIF, BTOR, or SMT-LIB utilized by formal verification solvers.⁶

While the open-source community relies on the free OSS CAD Suite for standard implementations, commercial environments and rigorous academic research frequently utilize the Tabby CAD Suite, an enterprise extension maintained by YosysHQ.² The Tabby CAD Suite integrates the proprietary Verific frontend, providing industry-grade support for complex SystemVerilog constructs, VHDL parsing, and SystemVerilog Assertions (SVA).² This dual-ecosystem approach ensures that Yosys remains accessible to hobbyists while meeting the stringent verification and synthesis demands of commercial semiconductor foundries.

Strategic Curriculum for Tool Mastery

Mastering a framework as complex and multifaceted as Yosys requires a phased, hierarchical approach that mirrors the synthesis pipeline itself. The learning architecture must be divided into sequential milestones, bridging high-level interactive design exploration with deep programmatic manipulation of the internal C++ codebase. This methodology ensures that an engineer understands not merely how to execute a synthesis script, but precisely how the logic graph is structurally mutated at every stage of the compilation process.

Mastery Phase	Pedagogical Focus Area	Core Commands and Utilities	Expected Analytical and Engineering Outcome
Phase I: Interactive Topology	Abstract Syntax Tree Generation, Basic Elaboration, and Visual Navigation	read_verilog, hierarchy, proc, show, write_json	The ability to manually elaborate structural HDL representations, resolve hierarchical parameters, and render logic graphs visually using GraphViz outputs. ⁹
Phase II: Algorithmic Optimization	Coarse-grain/Fine-grain synthesis separation, Boolean Minimization, Dead	opt, alumacc, share, clean, fsm, memory	A profound understanding of cyclic boolean optimization, finite

	Code Elimination		state machine recoding, and the extraction of arithmetic datapaths. ⁹
Phase III: Hardware Realization	Technology Mapping, Liberty File Integration, Delay Modeling, Switch-Level Abstraction	techmap, abc, abc9, dfflibmap, synth	Proficiency in mapping abstract mathematical operations to target-specific standard cell definitions (ASIC) or physical Lookup Tables (FPGA) while respecting delay parameters. ⁶
Phase IV: Mathematical Proofing	Bounded Model Checking (BMC), K-Induction, Satisfiability Solvers, Assertion Directives	sby, assert, assume, cover, GTKWave (.vcd traces)	The capacity to mathematically prove invariant safety and liveness properties within complex sequential designs using external SAT solvers. ¹²
Phase V: Internal Architecture	RTLIL API Mechanics, C++ Compilation, Custom Graph Traversal, Data Serialization	yosys-config, Pass inheritance, RTLIL::Design	The architectural capability to design bespoke compilation algorithms, such as specialized radiation-hardening logic injection or custom domain-specific optimizations. ²

To initiate this curriculum, users are strongly encouraged to bypass monolithic synthesis scripts initially and rely entirely on the interactive shell mode. Executing Yosys interactively allows hardware designers to feed commands incrementally, inspecting the intermediate netlist states

directly after every single graph transformation.⁹ This instantaneous feedback loop demystifies the complex transformation phases that proprietary commercial tools typically obscure behind opaque compilation logs.

Phase I: Interactive Design Exploration and AST Processing

The first phase of mastering the Yosys ecosystem involves understanding how human-readable hardware description languages are systematically ingested, verified, and translated into mathematical graph structures. When a user executes the `read_verilog` command (frequently utilizing the `-sv` flag to enable SystemVerilog features), the frontend parser initiates lexical analysis and parsing of the source file.⁹ This process constructs an Abstract Syntax Tree (AST), which is a hierarchical representation of the source code's grammatical structure.⁴

Once the AST is fully constructed in memory, Yosys executes the AST frontend compiler to generate the corresponding RTLIL representation.⁷ At this stage, the design is loaded, but it is not yet fully linked. The subsequent `hierarchy` command is critical; it analyzes the interconnected modules, resolves parameterized instantiations (such as generic widths passed to sub-modules), identifies the top-level entity using the `-top` argument, and prunes any unreferenced auxiliary modules from the working memory.⁹

A vital technique for beginners exploring the tool is manual elaboration paired with visual verification. Yosys integrates seamlessly with GraphViz via the `show` command, which renders the current in-memory RTLIL state as a graphical network map.⁹ Understanding the semiotics of these generated visualizations provides immediate insight into the tool's interpretation of the logic. Within these diagrams, octagonal boxes represent the primary input and output ports of the module. Rectangular boxes denote instantiated internal cells, which range from basic logic gates to complex arithmetic units. Ellipses are utilized to represent constant deterministic values driven into the logic network. The routing architecture is differentiated by line weight, where thin arrows indicate single-bit logical connections and thick arrows denote multi-bit bus wires.⁹ Furthermore, behavioral process blocks—which encapsulate sequential logic like always blocks or procedural if/case statements—are represented by large rounded-corner boxes labeled simply as PROC.⁹

By executing a manual elaboration sequence such as `proc; opt; show`, the user visually witnesses the translation of behavioral HDL into structural logic. The `proc` command dissects the abstract PROC boxes, converting procedural assignments and edge-sensitive triggers into physical multiplexer networks and generic \$dff (D-type flip-flop) cells.⁹ For interoperability with custom external analysis software, this precise hierarchical topology can be serialized and exported at any stage using the `write_json` command, which outputs a strictly formatted JSON file detailing every cell, port, net name, and attached attribute within the active design.⁹

Phase II: Algorithmic Equilibrium in Netlist Optimization

Synthesis optimization within Yosys functions across an abstraction gradient, categorized primarily into coarse-grain synthesis and fine-grain synthesis. Understanding the distinction is absolutely vital to deploying the correct heuristic algorithms at the proper stage of the compilation pipeline, ensuring that complex datapath structures are not prematurely destroyed by boolean flattening.¹⁰

Coarse-Grain Abstraction and Specialized Handling

When Verilog code is initially transformed from the AST into RTLIL, it exists in a highly abstract, coarse-grain state. Behavioral constructs and arithmetic operators exist as high-level entity blocks, represented by specialized internal cell types prefixed with a dollar sign (e.g., \$add, \$sub, \$mux, \$mem, \$div).¹⁰ Coarse-grain synthesis focuses on manipulating these macro-level structures before they are dissolved into individual single-bit logic gates. Preserving these high-level structures allows Yosys to analyze the broader mathematical intent of the circuit.

Several specialized commands are deployed exclusively during this coarse-grain phase. The fsm pass represents a highly sophisticated analysis engine that detects Finite State Machines (FSMs) embedded within the sequential logic.⁶ Once an FSM is identified, this pass extracts the state transition logic, performs reachability analysis to optimize unreachable states, and allows for the recoding of the state variables (for example, automatically converting a binary-encoded state machine into a physically faster one-hot encoding configuration).¹⁰ Similarly, the memory command suite handles the complex behavioral arrays defined in Verilog. It translates abstract array assignments into structured multi-port memory cells, paving the way for the memory_map pass to intelligently infer pattern structures like Simple Dual Port (SDP) or True Dual Port (TDP) RAM, and asymmetric memory configurations.¹⁰ Furthermore, the alumacc pass is invoked to extract and consolidate arithmetic logic. It identifies cascading adders, comparators, and multipliers, converting them into versatile \$alu and \$macc (multiply-accumulate) blocks. By isolating these mathematical pathways early, Yosys ensures they can eventually be mapped directly to highly efficient, dedicated Digital Signal Processing (DSP) slices on target FPGAs, rather than being inefficiently scattered across generic logic fabric.⁹

The Cyclic Dynamics of the opt Macro Architecture

Once specialized coarse-grain structures are resolved, the design undergoes intense logical reduction. The workhorse of both coarse and fine-grain optimization within Yosys is the opt macro command. Rather than executing a single, linear algorithm, opt operates as a composite controller. It iteratively calls a suite of specialized, targeted optimization sub-passes in a continuous loop until the netlist graph reaches a state of algorithmic equilibrium—defined as a full traversal where no further mathematical transformations or reductions are possible.⁹

An exhaustive understanding of the opt suite reveals the underlying mechanics of modern boolean logic simplification:

Optimization Pass	Algorithmic Function and Operational Mechanism
opt_expr	Executes constant folding and mathematical expression rewriting. It evaluates the netlist graph to identify logic cones driven entirely by static, deterministic inputs. For instance, if the logic dictates $x = (c \mid d) \& ((a \wedge b) \& 1'b0)$, the pass recursively deduces that the $\& 1'b0$ condition forces the entire right-hand expression to zero, subsequently propagating that zero backward to collapse the entire equation into a fixed 0 voltage driver, entirely removing the computational gates. ⁹
opt_merge	Performs trivial resource sharing by scanning the RTLIL graph for distinct cells that feature mathematically identical input stimulus and internal functional definitions. It merges these redundant operations into a single instantiated cell. By default, this pass avoids multiplexer cells (via the -nomux flag) to prevent the generation of dense, intertwined MUX trees that would obstruct downstream optimizations. ²¹
opt_muxtree	Identifies and eliminates dead branches within deeply nested multiplexer architectures. Behavioral HDL often generates multi-level if/else structures where specific control signal permutations are logically impossible. This pass prunes the logic cones feeding those inaccessible branches, isolating active paths. ¹⁰
opt_reduce	Targets expansive boolean networks, systematically simplifying overly large and

	unwieldy AND/OR gate networks and multiplexer trees into mathematically equivalent, tighter topologies. ¹⁰
opt_dff	Concentrates exclusively on sequential legalization, targeting D-type flip-flops (such as \$_DFF_, \$dff, and \$adff cells). It intelligently merges redundant clock enables and unifies synchronous reset multiplexers to eliminate unused control ports. If a flip-flop's data input is proven by opt_expr to be a static constant, the register itself is deleted and seamlessly replaced by a direct constant logical driver. ¹⁰
opt_clean	Operates as the systematic garbage collection mechanism. As preceding passes fold constants and sever dead branches, they leave behind disconnected wires and isolated, unread logic cells. opt_clean identifies these floating entities, applies an \unused_bits attribute for debugging, and physically excises them from the internal database to dramatically conserve memory footprint and reduce the computational overhead for downstream technology mapping. ²¹

A profound insight derived from analyzing Yosys's architectural approach is that synthesis optimization is a highly non-linear, chaotic system. The outputs generated by opt_muxtree frequently reveal previously obscured constant signals. These new constants mandate a subsequent, fresh run of opt_expr. The folding executed by opt_expr severs driving connections, which instantly creates a massive volume of dead wires requiring aggressive opt_clean garbage collection. The opt macro encapsulates this cyclic, cascading dependency inherently, executing while <changed design> is true.⁹

Phase III: Technology Mapping and the Switch-Level Abstraction

Following exhaustive boolean optimization, the highly simplified abstract network must be anchored to the physical realities and constraints of the target silicon architecture. This critical phase is defined as technology mapping. In comprehensive academic and industrial literature,

technology mapping spans a vast abstraction gradient, ranging from high-level coarse-grain cell substitution down through gate-level AIG mapping, and theoretically resting at the lowest physical foundation: switch-level modeling.¹⁰

Demystifying Switch-Level vs. Gate-Level Topologies

To fully comprehend the mapping pipeline, one must clearly distinguish between gate-level and switch-level abstraction. At the gate level, the logical elements instantiated are the pre-designed standard cells available in a foundry's Process Design Kit (PDK) or the specific physical Lookup Tables (LUTs) defined by an FPGA fabric architecture.¹¹ The synthesis tool views these cells as indivisible mathematical primitives.

Switch-level abstraction represents a deeper, fundamentally distinct, and highly granular physical modeling paradigm. In a switch-level representation, the entire digital circuit is not modeled as boolean logic gates (such as NAND, NOR, or XOR), but rather as a dense, interconnected netlist of individual, single complementary metal-oxide-semiconductor (CMOS) transistors operating as discrete electrical switches.¹¹

While switch-level modeling is syntactically possible in standard HDL languages like Verilog and VHDL, it is rarely, if ever, executed explicitly during the logic synthesis phase of modern ASIC or FPGA workflows.²³ Modern standard cell libraries are intricately pre-designed at the raw transistor layout level by dedicated analog layout engineers. These foundries optimize the individual transistor sizing, spacing, and doping to ensure hyper-precise timing, capacitance constraints, and thermal dissipation characteristics.¹⁶ Consequently, EDA logic synthesizers like Yosys relegate the physical switch-level architecture entirely to the Liberty (.lib) definition files provided by the foundry. The synthesis engine treats the standard gates defined within these files as immutable, atomic building blocks.¹¹ While Yosys does possess the capability to estimate baseline transistor requirements for generic CMOS implementations utilizing the `stat-tech cmos` diagnostic command (which outputs an estimated switch-level transistor count based on generic CMOS topologies), its actual synthesis engine operates firmly at the structural gate level to ensure timing models remain valid.¹¹

The Mechanics of the techmap Substitution Engine

To bridge the boundary between abstract RTL logic and technology-specific physical primitives, Yosys relies on the highly potent `techmap` command. Unlike complex heuristic mathematical mapping algorithms, `techmap` operates predominantly on a principle of structural architectural substitution. It functions as a macroscopic text-replacement and logic extraction engine driven by specifically crafted Verilog template patterns.⁹

When the `techmap` pass is invoked, it ingests predefined Verilog files containing specialized hardware implementations of abstract RTL cells, constructed from the target architecture's specific components.¹⁰ A highly unique and critical mechanism governing this behavior is the `(*techmap_celltype = "...")` attribute. When Yosys parses a technology map module containing this explicit attribute, it registers that module as an overriding translation template for the

designated cell type.²⁰ This methodology allows the toolchain to elegantly intercept high-level arithmetic operations—such as a complex \$mul operation followed immediately by an \$add operation—and map them directly into a highly specific FPGA DSP macro-block utilizing a sophisticated "wrap-extract-unwrap" sequence.¹⁰ This structural preservation ensures that complex, high-speed arithmetic hardware pathways are effectively utilized, preventing them from being destructively obliterated and flattened into thousands of inefficient logical NAND gates.¹⁰

ABC Integration for Multi-Level Logic Synthesis

For the fine-grain combinational logic mapping down to the foundational standard cell library, Yosys integrates seamlessly with the external, pre-existing academic tool ABC, developed and maintained by the University of California, Berkeley.¹¹ The synergistic integration between Yosys and the ABC optimizer represents the core algorithmic engine responsible for the toolchain's robust performance, allowing its Quality of Results (QoR) to achieve direct parity with established commercial alternatives.¹⁶

The integration process initiates when Yosys executes the abc (or the highly advanced abc9) pass. This command systematically extracts the combinational gate-level logic cloud from the RTLIL graph, reformats it into a compatible syntax, and passes it to the external ABC executable.¹¹ Within ABC, the logical functions are translated into highly optimized And-Inverter Graphs (AIGs) or Binary Decision Diagrams (BDDs).¹¹ ABC subsequently applies advanced multi-level logic synthesis algorithms, heavily relying on techniques such as structural choice evaluation and SAT sweeping.²⁸

SAT sweeping is a mathematically rigorous technique utilizing Boolean Satisfiability solvers to detect nodes within the AIG that are structurally disparate but functionally and mathematically equivalent.²⁸ Rather than simply discarding redundancies, ABC utilizes these equivalencies to derive structural choices, actively maintaining multiple parallel structural configurations of the exact same logic network simultaneously.²⁸ Utilizing precise timing models extracted from the foundry's Liberty file (passed via the abc -liberty directive), ABC dynamically calculates the signal propagation delay across these various AIG structures, selecting the optimal configuration based on whether the user is constraining the synthesis for maximum speed (delay reduction) or minimal area utilization.¹⁰

The abc9 implementation expands upon these capabilities drastically. It utilizes generalized, highly flexible delay models and incorporates a complete view of the circuit, actively acknowledging multiple distinct clock domains and accounting for the pathing of hard-blocks (like carry chains).¹⁰ This structural awareness allows Yosys-ABC9 to significantly reduce the critical path delay gap (achieving reductions from 30% down to 0% in specific FPGA targets for standard VTR benchmarks).²⁸ Depending on the architecture, ABC provides specialized commands such as lut2mux and lut2bmux to refine how lookup tables translate into multiplexer structures, maximizing the utilization of FPGA fabric.²⁹ Upon completing the extensive optimization and mapping, the refined technology-specific gate list is injected back into the

Yosys RTLIL structure, strictly preserving the original module hierarchy defined prior to extraction.²²

Sequential logic mapping, however, bypasses ABC entirely. It is managed natively within Yosys via the `dfflibmap` and `dfflegalize` passes.⁶ These commands parse the identical Liberty file to match RTLIL behavioral flip-flops with the exact physical sequential cells available in the foundry library. This involves meticulously matching hardware parameters, such as asynchronous set/reset configurations, clock edge polarities, and enable signal requirements.¹⁰ Developers mapping complex designs often encounter complexities with specialized sequential elements, such as latch mapping, requiring custom techmap overrides to correctly interface Yosys latch generation with proprietary library cells like the `TLATX1M`.³⁰

Phase IV: Mathematical Proofs and Formal Verification via SymbiYosys

While physical synthesis ensures the accurate structural translation of abstract logic to silicon domains, ensuring the functional safety and deterministic correctness of that logic is equally paramount, particularly in safety-critical applications. Within the open-source EDA ecosystem, this verification is executed utilizing SymbiYosys (SBY).¹² SBY operates as a sophisticated orchestration frontend that leverages Yosys purely for formal mathematical model extraction, bypassing physical synthesis. It pairs this extracted mathematical model with powerful external Satisfiability Modulo Theories (SMT) and Boolean Satisfiability (SAT) solvers, such as Boolector, Z3, or Yices.³

Formal verification fundamentally disrupts traditional validation flows by systematically eliminating the reliance on a validation engineer's imagination to write exhaustive, cycle-by-cycle test benches.³² Instead, it uses rigorous mathematical solvers to traverse the entirety of the logic design's state-space matrix. It validates whether a declared set of rules (properties) can ever be violated under *any conceivable combination* of input permutations across infinite time.³²

Constructing Formal Properties in Verilog

To establish the parameters of verification, extensions to the Verilog standard must be defined directly within the RTL code, or bound via SystemVerilog Assertions (SVA).¹² Within the Tabby CAD Suite, the Verific frontend natively parses complex SVA directives, while the open-source frontend handles fundamental immediate assertions seamlessly.⁵ Yosys accommodates three core immediate assertion constructs to guide the SMT solvers¹²:

Formal Directive	Mathematical Application and Solver Interpretation
------------------	--

assume(<expr>;	An assumption acts as a rigid mathematical constraint applied to the input search space. It informs the SMT solver that specific input scenarios are physically impossible within the overarching system architecture. This forces the solver to aggressively prune those branches from its matrix search. For example, assuming an active-low hardware reset signal will never logically toggle concurrently with a high-priority software interrupt drastically reduces the required search space. ¹²
assert(<expr>;	Assertions dictate the absolute laws the logic design is obligated to uphold. The solver's fundamental objective is to act as an adversary, attempting mathematically to render the assertion expression false. If the solver discovers an input sequence that violates an assertion, the formal proof fails, indicating a critical design flaw. ¹²
cover(<expr>;	Rather than seeking failure conditions, coverage dictates reachability and liveness. A cover statement demands that the solver discover at least one valid path traversing the complex state machine to reach a specified operational condition. This validates that intended functional states are actually accessible and not blocked permanently by dead logic paths. ¹²

A standard pedagogical example utilized to teach SymbiYosys involves verifying a First In, First Out (FIFO) buffer.¹² In this architecture, an `addr_gen` module handles read and write addresses, incrementing pointers and wrapping back to zero to create a circular queue mechanism within a register file.¹² To verify this, engineers construct properties such as `a_oflow` (asserting that the data count never mathematically exceeds the total available memory array, `count <= MAX_DATA`).¹²

Furthermore, in sequential property checking, analyzing state transitions requires comparing the present cycle against previous cycles. This is achieved using the `$past()` function. An assertion like `a_counts` utilizes `$past` to guarantee that a buffer count only ever increments by 1, decrements by 1, or remains static, ensuring that the pointer logic cannot jump erratically due

to an integer overflow error.¹² Complex concurrent assertions, such as a `write_skip` property monitoring a sequence where a low write-enable signal is followed strictly by a change in write address, are optimally handled utilizing SVA bindings.¹²

Bounded Model Checking and Unbounded k -Induction

SymbiYosys provides an array of specific operational modes dictated within the `[options]` section of the `.sby` configuration script, predominantly focusing on Bounded Model Checking (BMC) and Mathematical Induction (`prove` mode).¹²

When executing in `bmc` mode, the SAT solver performs a technique known as loop unrolling. It unfolds the sequential logic into a massive, monolithic combinational logic equation extending exactly k clock cycles deep (where k is an arbitrary user-defined temporal bound, e.g., 20 cycles).¹² It initiates the simulation at the global reset state and searches step-by-step for any vulnerability that breaks an assert. However, passing a BMC test is fundamentally inconclusive for continuous operational safety; it merely proves that the design does not experience a critical failure within the first k cycles of booting.¹²

To achieve a mathematically rigorous, unbounded, infinite-time proof, SBY utilizes the `prove` mode, invoking the principle of k -induction. The algorithm first initiates a standard BMC run to serve as the mathematical base case. Subsequently, it attempts the inductive step: the solver forcefully posits that the state machine is currently sitting in a completely arbitrary state at an arbitrary timestep n , and assumes all assertions are currently valid in this state. The solver then simulates cycle $n + 1$. If an assertion fails at step $n + 1$, the proof yields an UNKNOWN or FAIL result.¹³ A critical design scenario frequently occurs where a design fails induction but passes BMC. This almost exclusively indicates that the solver initiated the inductive step from an "unreachable" state—a ghost state that the physical hardware logic could never naturally navigate into from reset.¹³ Mitigating this requires the verification engineer to craft deeper, more precise assume constraints to prevent the solver from exploring states the hardware architecture forbids.¹³

Upon locating a violation, SymbiYosys automatically synthesizes a Value Change Dump (VCD) trace file representing the exact, cycle-by-cycle input waveforms that precipitated the failure. Developers can then load this counterexample visually within the GTKWave interface, analyzing the discrepancy (e.g., observing the precise cycle where a data counter diverges from the physical address difference during an underfill condition) to rapidly patch the underlying RTL.¹²

Phase V: Internal Architecture and the RTL Intermediate Language (RTLIL)

Achieving true mastery over the Yosys infrastructure dictates moving beyond the execution of

external user scripts and comprehensively understanding its internal programmatic data formats. At the absolute nucleus of the entire software architecture resides the RTL Intermediate Language (RTLIL).⁷ All frontends—regardless of whether they process Verilog, VHDL, or structural netlists—inevitably deposit their parsed data into RTLIL representations. Consequently, all subsequent analysis passes and manipulation logic interface directly, and only, with this strictly defined graph structure.⁴

The Programmatic Entity-Relationship Hierarchy of RTLIL

Conceptually, the Yosys framework retains exactly one active "current design" in memory at any given time, though specific passes may spawn auxiliary, isolated instances internally to safely parse external library formats without polluting the primary namespace.⁸ The C++ data structures encapsulate a rigid, multi-layered Entity-Relationship hierarchy, constructed to model the physical reality of a hardware circuit⁸:

RTLIL C++ Class	Structural Function and Hardware Representation
RTLIL::Design	The absolute root object of the data structure. It contains global configurations, tracking metadata, and an indexed map pointing to all distinct modules existing within the bounds of the circuit. ⁸
RTLIL::Module	Corresponding directly to Verilog modules or VHDL entities, this container houses the discrete logic structures. Within a single module, there exist N instances of various structural subclasses. ⁸
RTLIL::Cell	Representations of instantiated operational hardware blocks. These range from simple internal library primitives (e.g., \$and, \$mux) to massive, complex instances of other entire modules. Cells are rigidly defined by their internal parameters and port connections. ⁸
RTLIL::Wire	The mathematical nodes that connect disparate elements of the circuit. Wires encompass comprehensive metadata, including vector width, bit directionality

	(input, output, or inout), and numeric signedness formatting. ⁸
RTLIL::Memory	Specialized multidimensional array constructs engineered for handling dense RAM/ROM array initializations prior to the memory-mapping optimization phases. ⁸
RTLIL::Process	High-level sequential behavior blocks that translate synchronous edge sensitivities and procedural if/else multiplexer logic into register state updates. Processes consist of SyncRule definitions to dictate trigger events (e.g., posedge clk, sync init) and SwitchRule definitions governing combinatorial branching pathways. ³⁴

The RTLIL Text Representation Format Specification (.il)

To facilitate low-level debugging, serialization, and interaction with entirely external modern toolchains (such as integrating Yosys within MLIR/CIRCT architectures), the in-memory RTLIL C++ structures map bidirectionally to a plain-text serialized format. This format is commonly designated with an .il extension and follows a strict, highly parsable Extended Backus-Naur Form (EBNF) grammar specification.³⁵

The serialization file initializes with an autoidx declaration. This integer statement provides a global index baseline, ensuring that randomly generated names for internal wires and cells (e.g., flatten15) remain consistently unique and deterministic across successive optimization passes.³⁵ Without autoidx, dumping a design mid-compilation and resuming it later would result in index collisions, destabilizing the graph.³⁵

Identifiers within the .il format are categorically split into two distinct namespaces. Public identifiers, prefixed by a backslash (\), reflect explicit user definitions instantiated in the original HDL source code (e.g., \reset_n, \data_bus). Conversely, auto-generated internal identifiers are prefixed by a dollar sign (\$), representing structures synthesized entirely by the toolchain (e.g., \$proc_0, \$abc\$123).³⁵ Values and integers are strictly typed; an integer constant defined as a sigspec is perpetually treated as a 32-bit wide, two's complement value unless explicitly overridden.³⁵ Bits are designated not merely as 0 or 1, but accommodate logic states including x (unknown), z (high-impedance), m (marked), or - (don't care).³⁵

A critical foundational concept within the RTLIL architecture is the RTLIL::SigSpec (Signal Specification).³⁵ A SigSpec is not inherently a wire itself; rather, it acts as a flexible mathematical descriptor that binds physical cell ports to specific data sources. A SigSpec can represent a constant deterministic value (e.g., 8'11001010), a single discrete bit extracted dynamically from

a wider bus (e.g., `\bus_data`), an entire wire, or a complex concatenation of various disparate signal sources enclosed in brackets.³⁵

A concrete text representation of an RTLIL cell mapping a standard 2-input boolean AND operator highlights the precise connectivity semantics governed by SigSpec parameters³⁵:

Code snippet

```
cell $and \my_and_gate
  parameter \A_WIDTH 8
  parameter \B_WIDTH 8
  parameter \Y_WIDTH 8
  connect \A 8'00001111
  connect \B \data_out
  connect \Y \result
end
```

By learning to read and write the serialized .il format directly, researchers can precisely pinpoint the structural modifications inflicted by specific passes during synthesis without wrestling with abstract C++ graph traversal algorithms, making it an invaluable diagnostic tool.³⁵ Furthermore, this intermediate representation is precisely the mechanism utilized when interfacing Yosys with external MLIR (Multi-Level Intermediate Representation) dialects, such as the CIRCT project, facilitating advanced hardware optimization specifically tailored for cryptographic workloads.³⁶ This dual-licensing compatibility (ISC and LLVM exceptions) ensures seamless integration across modern compiler ecosystems.³⁶

Phase VI: C++ Extensibility and Academic Integration

The most potent and revolutionary characteristic of Yosys is its status as a foundational, open-source C++ framework, granting users absolute transparency and programmatic control over the synthesis graph. For EDA researchers developing unconventional hardware flows—such as domain-specific logic implementations, structural hardware trojan injection, or the automated mapping of complex redundancy architectures for the aerospace sector—commercial proprietary tools are notoriously inflexible, actively prohibiting low-level netlist manipulation.¹⁵ Yosys rectifies this architectural roadblock by exposing an API that allows for the creation of native, dynamically loadable plugins. For developers preferring higher-level scripting, Pyosys exposes a limited subset of the C++ API directly to the Python programming language, allowing for object-oriented scripting and advanced control flow over the database.³⁸

Structural Scaffolding of a Custom C++ Pass

All procedural passes and backends injected into Yosys trace their inheritance to the generic Pass or ScriptPass base classes located within the RTLIL namespace.² A custom plugin fundamentally consists of defining a subclass, registering a unique command string within the constructor, and overriding the virtual execute() method.⁴⁰

To demonstrate the baseline architecture, consider the rudimentary "Hello World" pass syntax universally utilized as a developmental starting point for writing Yosys extensions¹⁶:

C++

```
#include "kernel/yosys.h"
```

```
USING_YOSYS_NAMESPACE  
PRIVATE_NAMESPACE_BEGIN
```

```
struct HelloWorldPass : public Pass {  
    HelloWorldPass() : Pass("hello_world") { }  
  
    void execute(vector<string>, Design*) override {  
        log("Hello World Execution Initialized!\n");  
    }  
} HelloWorldPass;
```

```
PRIVATE_NAMESPACE_END
```

The application of the PRIVATE_NAMESPACE_BEGIN macro is fundamentally critical to the stability of the software. It walls off the internal data structures and variable names of the custom plugin from unpredictable collisions within the broader global Yosys namespace architecture.¹⁶

Compilation of custom plugins avoids manual dependency resolution by relying on a specialized configuration script bundled seamlessly with the suite: yosys-config. This utility automatically inspects the environment and injects the exact library linkages, compiler variables (--cxx, --cxxflags), and headers required to interface seamlessly with the installed binaries. Executing a command such as yosys-config --build hello.so hello.cc automatically produces a shared object that can be dynamically injected into any interactive Yosys shell session or synthesis script using the plugin -i hello.so command.¹⁶

Interacting with the Netlist Database: The TaMaRa Example

Developing meaningful algorithmic transformations demands interacting deeply with the RTLIL::Design pointer provided to the execute() function during initialization. The process begins with traversing the graph hierarchy. Developers must iterate over the specific modules present within the overarching system to locate target logic.²

A standard architectural pattern used universally across the Yosys codebase involves a primary for loop extracting active modules, followed by a secondary nested traversal iterating through cells() or wires().² A prime example of this applied in academic research is the TaMaRa algorithm, designed to automate Triple Modular Redundancy (TMR) for radiation-hardened designs targeting environments subject to Single Event Upsets (SEUs).¹⁵ The custom C++ code iterates across the netlist targeting RTLIL::Wire and RTLIL::Cell objects. The logic dynamically extracts standard sequential flip-flops and reconstructs three identical, parallel instantiations (a triplication pass). It then physically severs the original wire routes and re-routes the resulting signals into a newly defined, custom voter cell instantiated entirely on-the-fly via the C++ API.¹⁵ Tests simulating fault injection demonstrate that unprotected circuits fail almost immediately, whereas the TaMaRa TMR mitigated circuits drastically improve functional reliability.¹⁵

When manipulating the graph dynamically during these operations, structural and memory integrity must be prioritized. It is generally unsafe to haphazardly delete signal wires or cell objects mid-traversal, as it risks corrupting the iteration index pointers and causing memory segmentation faults.³⁹ Instead, robust passes orchestrate changes by disconnecting unwanted logic paths and appending specific attribute metadata strings (e.g., \unused) directly to the dangling ports. Following the traversal, the pass then invokes the standard opt_clean command programmatically to handle safe, deep-level garbage collection.¹⁰

Furthermore, architectural best practices dictate that a single custom pass should avoid excessive complexity.³⁹ If a developer aims to craft a top-level command that requires both custom graph manipulation and the invocation of pre-existing Yosys optimizations (such as initiating a constant fold halfway through a traversal to evaluate a new logic cone), they should subclass the ScriptPass foundation rather than the raw Pass framework.³⁹ The ScriptPass includes specialized control flow mechanisms explicitly designed to execute external script-like routines, isolating the graph manipulation logic tightly into constrained, easily debuggable functions.¹⁶

Application in Pedagogy and Democratic Hardware Access

The maturation of Yosys, its formalized documentation, and its robust APIs have precipitated a transformation in how computer engineering is taught at the university level. Historically, teaching hardware design to hundreds of students required relying on highly complex, opaque commercial vendor tools—characterized critically by academic researchers as "Point and Click Adventure Games" due to their reliance on convoluted GUI interactions and massive, fragile

installation footprints.⁴¹

Institutions like the University of California, Santa Cruz (UCSC) are actively mitigating this workforce scaling bottleneck by integrating the open-source EDA toolchain directly into core computer engineering series.⁴¹ The inherent transparency of Yosys allows students to peer underneath the abstraction layers, observing exactly how a behavioral assignment collapses into a generic logic gate, and subsequently how that gate is mapped to physical silicon. Furthermore, Yosys acts as the foundational compilation engine for prominent educational and hobbyist architectures, integrating seamlessly with the LiteX framework (a Python-based System-on-Chip assembly builder) to deploy minimalist open-source processor cores like the FemtoRV, picoRV, and SERV directly onto highly accessible sub-\$50 FPGA development boards like the Lattice IceStick.⁴² By completely eliminating the licensing friction and installation complexity inherent to commercial tools, Yosys ensures that exploring RISC-V processor design and pipelined architectures remains an accessible, purely educational endeavor rather than an exercise in software administration.⁴¹

The exhaustive exploration and utilization of the Yosys Open SYnthesis Suite represents a critical evolution from constrained, linear hardware logic design to a multidimensional, fully programmable silicon synthesis architecture. Through methodical mastery of the diverse system components—ranging from leveraging macro optimization techniques in the opt suite to manipulating fine-grain mathematical boolean matrices utilizing the ABC external integration—engineers gain the unparalleled ability to drastically reduce logic area utilization and critical path delay. Integrating Formal Verification methodologies through SymbiYosys ensures the functional, deterministic viability of the physical silicon map long before the expensive, time-consuming layout stages begin. Finally, by transitioning beyond mere script generation into raw C++ plugin formulation and RTLIL graph manipulation, academics and industry professionals alike unlock a mechanism to pioneer entirely novel domain-specific hardware synthesis pipelines, fundamentally proving that open-source mathematical toolchains are capable of performing on par with, and often uniquely exceeding the capabilities of, proprietary corporate synthesis frameworks.

Works cited

1. YosysHQ Yosys, accessed on May 9, 2026, https://yosyshq.readthedocs.io/_/downloads/yosys/en/0.24/pdf/
2. FPGA Ignite Yosys Course 2023-08-02, accessed on May 9, 2026, https://fpga-ignite.github.io/fpga_ignite_2023/yosys/FPGA%20Ignite%20Yosys%20Course%202023-08-02-3.pdf
3. What is Yosys - YosysHQ Docs documentation, accessed on May 9, 2026, <https://yosyshq.readthedocs.io/projects/yosys/en/stable/introduction.html>
4. 3. Approach - YosysHQ Docs, accessed on May 9, 2026, https://yosyshq.readthedocs.io/projects/yosys/en/0.27/CHAPTER_Approach.html
5. YosysHQ/yosys: Yosys Open SYnthesis Suite - GitHub, accessed on May 9, 2026, <https://github.com/yosyshq/yosys>

6. Yosys Open SYnthesis Suite :: About - YosysHQ, accessed on May 9, 2026, <https://yosyshq.net/yosys/>
7. Implementation Overview — Yosys documentation, accessed on May 9, 2026, https://blog.eowyn.net/yosys/CHAPTER_Overview.html
8. The RTL Intermediate Language (RTLIL) - YosysHQ Yosys 0.64-dev documentation, accessed on May 9, 2026, https://yosyshq.readthedocs.io/projects/yosys/en/latest/yosys_internals/formats/rtlil_rep.html
9. open-eda-course/yosys-tutorial/yosys-tutorial.md at main - GitHub, accessed on May 9, 2026, <https://github.com/asinghani/open-eda-course/blob/main/yosys-tutorial/yosys-tutorial.md>
10. Synthesis in detail - YosysHQ Yosys 0.64-dev documentation, accessed on May 9, 2026, https://yosyshq.readthedocs.io/projects/yosys/en/latest/using_yosys/synthesis/index.html
11. Yosys Manual - GitHub, accessed on May 9, 2026, https://raw.githubusercontent.com/wiki/jospicant/IceStudio/yosys_manual.pdf
12. Getting started - YosysHQ SBY documentation, accessed on May 9, 2026, <https://symbiosys.readthedocs.io/en/latest/quickstart.html>
13. An Exercise in using Formal Induction - ZipCPU, accessed on May 9, 2026, <https://zipcpu.com/blog/2018/03/10/induction-exercise.html>
14. Formal extensions to Verilog - YosysHQ SBY documentation, accessed on May 9, 2026, <https://symbiosys.readthedocs.io/en/latest/verilog.html>
15. TaMaRa: Towards a Triple Modular Redundancy Pass for Yosys - YosysHQ Blog, accessed on May 9, 2026, <https://blog.yosyshq.com/p/tamara-towards-a-triple-modular-redundancy-pass-for-yosys/>
16. YosysHQ Yosys, accessed on May 9, 2026, https://yosyshq.readthedocs.io/_/downloads/yosys/en/0.34/pdf/
17. How to disable optimizations in Yosys synthesis script and ABC mapping of cells? : r/FPGA - Reddit, accessed on May 9, 2026, https://www.reddit.com/r/FPGA/comments/1mwo1ej/how_to_disable_optimizations_in_yosys_synthesis/
18. Synthesis with Yosys - PULP Platform, accessed on May 9, 2026, https://pulp-platform.org/docs/efclseminar2026/track1_ex/ex03.pdf
19. Internal formats - YosysHQ Yosys 0.64-dev documentation, accessed on May 9, 2026, https://yosyshq.readthedocs.io/projects/yosys/en/latest/yosys_internals/formats/index.html
20. Logic Primitive Transformations with Yosys Techmap - Tom Verbeure, accessed on May 9, 2026, <https://tomverbeure.github.io/2022/11/18/Primitive-Transformations-with-Yosys-Techmap.html>
21. Optimization passes - YosysHQ Yosys documentation, accessed on May 9, 2026,

- https://yosyshq.readthedocs.io/projects/yosys/en/0.40/using_yosys/synthesis/opt.html
22. Optimizations — Yosys documentation, accessed on May 9, 2026,
https://blog.eowyn.net/yosys/CHAPTER_Optimize.html
 23. A primer on digital circuit synthesis - YosysHQ Yosys 0.64 documentation, accessed on May 9, 2026,
<https://yosyshq.readthedocs.io/projects/yosys/en/stable/appendix/primer.html>
 24. A primer on digital circuit synthesis - YosysHQ Yosys 0.64-dev documentation, accessed on May 9, 2026,
<https://yosyshq.readthedocs.io/projects/yosys/en/latest/appendix/primer.html>
 25. Yosys: Getting Gates /Transistor count - Stack Overflow, accessed on May 9, 2026,
<https://stackoverflow.com/questions/56545305/yosys-getting-gates-transistor-count>
 26. 9. Technology mapping - YosysHQ Docs documentation, accessed on May 9, 2026,
https://yosyshq.readthedocs.io/projects/yosys/en/0.37/CHAPTER_Techmap.html
 27. Yosys manual - YosysHQ Docs, accessed on May 9, 2026,
<https://yosyshq.readthedocs.io/projects/yosys/en/0.33/>
 28. Academic FPGA Synthesis is Catching Up With the Industry - People @EECS - University of California, Berkeley, accessed on May 9, 2026,
http://people.eecs.berkeley.edu/~alanmi/publications/2023/date23_gap.pdf
 29. Command line reference - YosysHQ Yosys 0.63 documentation, accessed on May 9, 2026, https://yosyshq.readthedocs.io/projects/yosys/en/v0.63/cmd_ref.html
 30. How to create a custom technology cell map for Yosys - Stack Overflow, accessed on May 9, 2026,
<https://stackoverflow.com/questions/60143268/how-to-create-a-custom-technology-cell-map-for-yosys>
 31. Exploring Logic Synthesis with Yosys: A Tutorial Overview | by Srinivas Rahul Sapireddy, accessed on May 9, 2026,
<https://srsapireddy.medium.com/exploring-logic-synthesis-with-yosys-a-tutorial-overview-1d8f63783d86>
 32. Yosys and SymbiYosys - ISEC, accessed on May 9, 2026,
<https://www.isec.tugraz.at/wp-content/uploads/2020/07/yosys.pdf>
 33. Internal formats - YosysHQ Yosys documentation, accessed on May 9, 2026,
https://yosyshq.readthedocs.io/projects/yosys/en/0.40/yosys_internals/formats/
 34. Yosys Open Source Synthesis Overview | PDF - Scribd, accessed on May 9, 2026,
<https://www.scribd.com/document/686274864/Yosis-Presentation>
 35. RTLIL text representation - YosysHQ Yosys documentation, accessed on May 9, 2026,
https://yosyshq.readthedocs.io/projects/yosys/en/0.40/yosys_internals/formats/rtlil_text.html
 36. YosysHQ/rtlil-mlir: Yosys RTLIL dialect for MLIR - GitHub, accessed on May 9, 2026, <https://github.com/YosysHQ/rtlil-mlir>
 37. Integration with ABC optimizer - CIRCT - LLVM Discussion Forums, accessed on

- May 9, 2026, <https://discourse.llvm.org/t/integration-with-abc-optimizer/65264>
38. yosys/docs/source/using_yosys/pyosys.rst at main · YosysHQ/yosys - GitHub, accessed on May 9, 2026, https://github.com/YosysHQ/yosys/blob/main/docs/source/using_yosys/pyosys.rst?plain=true
 39. YosysHQ Yosys, accessed on May 9, 2026, https://yosyshq.readthedocs.io/_/downloads/en/0.33/pdf/
 40. Programming Yosys Extensions, accessed on May 9, 2026, https://blog.eowyn.net/yosys/CHAPTER_Prog.html
 41. Community Spotlight - Teaching with Open Source Tools - YosysHQ Blog, accessed on May 9, 2026, <https://blog.yosyshq.com/p/community-spotlight-teaching-with-open-source-tools/>
 42. BrunoLevy/learn-fpga: Learning FPGA, yosys, nextpnr, and RISC-V - GitHub, accessed on May 9, 2026, <https://github.com/BrunoLevy/learn-fpga>